

Pandas — LEVEL 8: Filtering Data

Very important for Data Analyst / Data Science — line-by-line code, outputs, examples, and interview explanations.

1. Comparison Operators

Code

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John", "Neha"],
    "Age": [22, 25, 30, 20],
    "Salary": [30000, 45000, 60000, 25000]
})

print(df)
```

Output

	Name	Age	Salary
0	Amit	22	30000
1	Riya	25	45000
2	John	30	60000
3	Neha	20	25000

Explanation: Comparison operators compare values and return True or False.

Easy interview explanation: Filtering starts by creating a Boolean condition.

2. Greater Than >

Code

```
result = df[df["Age"] > 23]

print(result)
```

Output

	Name	Age	Salary
1	Riya	25	45000
2	John	30	60000

Explanation: Selects rows where Age is greater than 23.

Easy interview explanation: The > operator means greater than.

3. Less Than <

Code

```
result = df[df["Age"] < 25]

print(result)
```

Output

	Name	Age	Salary
0	Amit	22	30000
3	Neha	20	25000

Explanation: Selects rows where Age is less than 25.

Easy interview explanation: The < operator means less than.

4. Greater Than or Equal >=

Code

```
result = df[df["Age"] >= 25]

print(result)
```

Output

	Name	Age	Salary
1	Riya	25	45000
2	John	30	60000

Explanation: Includes 25 and values greater than 25.

Easy interview explanation: The >= operator means greater than or equal to.

5. Less Than or Equal <=

Code

```
result = df[df["Age"] <= 22]

print(result)
```

Output

	Name	Age	Salary
0	Amit	22	30000
3	Neha	20	25000

Explanation: Includes 22 and values below 22.

Easy interview explanation: The <= operator means less than or equal to.

6. Equal ==

Code

```
result = df[df["Age"] == 25]

print(result)
```

Output

	Name	Age	Salary
1	Riya	25	45000

Explanation: Selects rows where Age exactly equals 25.

Easy interview explanation: Use == for comparison; = is assignment.

7. Not Equal !=

Code

```
result = df[df["Age"] != 25]

print(result)
```

Output

	Name	Age	Salary
0	Amit	22	30000
2	John	30	60000
3	Neha	20	25000

Explanation: Selects rows where Age is not 25.

Easy interview explanation: The `!=` operator means not equal.

8. Boolean Conditions

Code

```
condition = df["Age"] > 23

print(condition)
```

Output

```
0    False
1     True
2     True
3    False
Name: Age, dtype: bool
```

Explanation: A Boolean condition gives True or False for every row.

Easy interview explanation: True rows are selected with `df[condition]`.

9. AND &

Code

```
result = df[(df["Age"] > 20) & (df["Salary"] > 30000)]

print(result)
```

Output

	Name	Age	Salary
1	Riya	25	45000
2	John	30	60000

Explanation: `&` means AND. Both conditions must be True.

Easy interview explanation: Always put parentheses around each condition when using `&`.

10. OR |

Code

```
result = df[(df["Age"] < 21) | (df["Salary"] > 50000)]

print(result)
```

Output

	Name	Age	Salary
2	John	30	60000
3	Neha	20	25000

Explanation: `|` means OR. At least one condition must be True.

Easy interview explanation: Always put parentheses around each condition when using `|`.

11. NOT ~

Code

```
result = df[~(df["Age"] > 23)]

print(result)
```

Output

```
   Name  Age  Salary
0  Amit   22   30000
3  Neha   20   25000
```

Explanation: ~ reverses a Boolean condition.

Easy interview explanation: Use ~ for the opposite of a condition.

12. Multiple Conditions

Code

```
result = df[
    (df["Age"] >= 22) &
    (df["Salary"] >= 30000) &
    (df["Name"] != "John")
]

print(result)
```

Output

```
   Name  Age  Salary
0  Amit   22   30000
1  Riya   25   45000
```

Explanation: Several conditions can be combined using & and |.

Easy interview explanation: Use parentheses around every condition.

13. isin()

Code

```
result = df[df["Name"].isin(["Amit", "Neha"])]

print(result)
```

Output

```
   Name  Age  Salary
0  Amit   22   30000
3  Neha   20   25000
```

Explanation: isin() checks whether each value exists in a list.

Easy interview explanation: isin() is useful for filtering multiple exact values.

14. between()

Code

```
result = df[df["Age"].between(22, 25)]

print(result)
```

Output

```
   Name  Age  Salary
0  Amit   22   30000
1  Riya   25   45000
```

Explanation: `between(22, 25)` selects values from 22 through 25; both ends are included by default.

Easy interview explanation: `between()` is useful for numeric ranges and dates.

15. query()

Code

```
result = df.query("Age > 23 and Salary > 40000")

print(result)
```

Output

```
   Name  Age  Salary
1  Riya   25   45000
2  John   30   60000
```

Explanation: `query()` lets you write a readable filtering expression as a string.

Easy interview explanation: `query()` can make complex filters easier to read.

16. query() with OR

Code

```
result = df.query("Age < 21 or Salary > 50000")

print(result)
```

Output

```
   Name  Age  Salary
2  John   30   60000
3  Neha   20   25000
```

Explanation: Inside `query()`, use words such as `and` and `or`.

Easy interview explanation: `query()` uses an expression string rather than `&` and `|` syntax.

17. Filter Text

Code

```
result = df[df["Name"] == "Amit"]

print(result)
```

Output

```
   Name  Age  Salary
0  Amit   22   30000
```

Explanation: Text values can be filtered by comparing them with a string.

Easy interview explanation: Text filtering is very common in Data Analyst work.

18. Filter by Salary

Code

```
result = df[df["Salary"] >= 40000]

print(result)
```

Output

```
   Name  Age  Salary
1  Riya   25   45000
2  John   30   60000
```

Explanation: Selects employees whose salary is at least 40,000.

Easy interview explanation: Numeric filtering is one of the most common DataFrame operations.

19. Real-World Combined Example

Code

```
result = df[
    (df["Age"].between(22, 30)) &
    (df["Name"].isin(["Amit", "John", "Riya"]))
]

print(result)
```

Output

```
   Name  Age  Salary
0  Amit   22   30000
1  Riya   25   45000
2  John   30   60000
```

Explanation: `between()` and `isin()` are combined with `&` to create a practical filter.

Easy interview explanation: Combined filtering is frequently used in real Data Analyst projects.

20. Quick Revision

Code

```
# Greater than
df[df["Age"] > 23]

# Less than
df[df["Age"] < 25]

# Equal
df[df["Age"] == 25]

# Not equal
df[df["Age"] != 25]

# AND
df[(df["Age"] > 20) & (df["Salary"] > 30000)]

# OR
df[(df["Age"] < 21) | (df["Salary"] > 50000)]

# NOT
df[~(df["Age"] > 23)]

# Multiple values
df[df["Name"].isin(["Amit", "Neha"])]

# Range
df[df["Age"].between(22, 25)]
```

```
# Query
df.query("Age > 23 and Salary > 40000")
```

Output

Comparison:

> < >= <= == !=

Logical:

& -> AND

| -> OR

~ -> NOT

Functions:

isin() -> match values from a list

between() -> filter a range

query() -> readable filter expression

Explanation: These are the core pandas filtering techniques for Data Analyst and Data Science work.

Easy interview explanation: Most important rule: use parentheses around conditions with & and |.